



Instituto Tecnológico Superior de Lerdo

DESARROLLO DE APLICACIONES PARA DISPOSITIVOS MÓVILES UNIDAD 4

Actividad 4.1 - Creación de un Mini Reproductor de Música en Flutter

Nombre del Alumno(s)

Jesús Antonio Arellano Zapata

Número de Control

202310117

Docente:

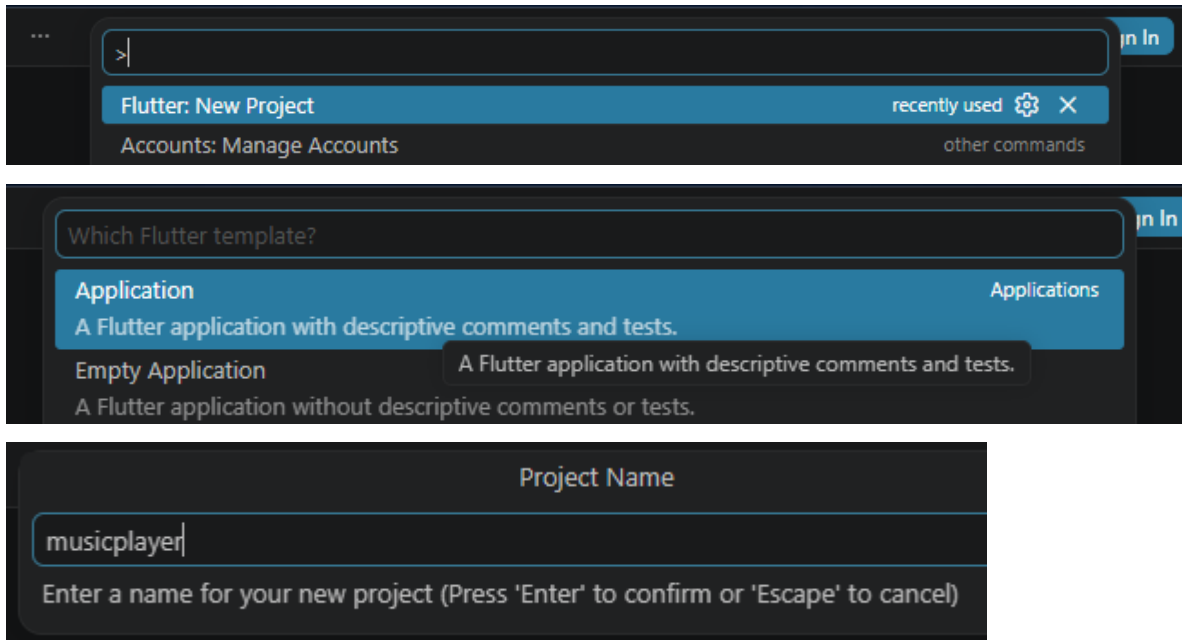
Jesús Salas Marín

Fecha de Entrega

12/06/2026

Creando el proyecto Flutter

En este paso creé un nuevo proyecto de Flutter desde Visual Studio Code, utilizando la opción Flutter: New Project y seleccionando la plantilla Application. Después asigné el nombre del proyecto como musicplayer, para comenzar el desarrollo del mini reproductor de música.



Agregando los archivos de audio e imagen

En este paso agregué la carpeta assets dentro del proyecto, donde coloqué el archivo de audio baby.mp3 y la imagen justin.jpg. Estos archivos serán utilizados por la aplicación para reproducir la canción y mostrar la portada del reproductor.



Configurando las dependencias del proyecto

En este paso modifiqué el archivo pubspec.yaml, agregando las dependencias just_audio y rxdart. Estas librerías permiten reproducir audio y manejar los datos de posición, duración y progreso de la canción.

```
dependencies:  
  flutter:  
    sdk: flutter  
  just_audio: ^0.9.36  
  rxdart: ^0.27.7
```

Registrando los assets en el proyecto

En este paso agregué las rutas de los archivos dentro de pubspec.yaml, para que Flutter pudiera reconocer el audio y la imagen que se usarán en la aplicación.

```
assets:  
  - assets/baby.mp3  
  - assets/justin.jpg
```

Creando el archivo principal main.dart

En este paso configuraré el archivo main.dart, donde se ejecuta la aplicación mediante runApp() y se crea la clase principal MusicPlayerApp. También se configuró MaterialApp para iniciar la pantalla del reproductor de música.

```
lib > main.dart > MusicPlayerApp > build
1  import 'package:flutter/material.dart';
2
3  void main() {
4    runApp(const MusicPlayerApp());
5  }
6
7  class MusicPlayerApp extends StatelessWidget {
8    const MusicPlayerApp({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return MaterialApp(
13       debugShowCheckedModeBanner: false,
14       title: 'Music Player',
15
16       home: const MusicPlayerScreen(),
17     ); // MaterialApp
18   }
19 }
```

Creando la clase PositionData

En este paso creé el archivo position_data.dart, donde se definió la clase PositionData. Esta clase guarda la posición actual, la posición almacenada en búfer y la duración total de la canción.

```
lib > position_data.dart > PositionData > PositionData
1  class PositionData {
2    final Duration position;
3    final Duration bufferedPosition;
4    final Duration duration;
5
6    PositionData(this.position, this.bufferedPosition, this.duration);
7  }
```

Creando el Widget SeekBar

En este paso creé el archivo seek_bar.dart, donde se definió el widget SeekBar. Este widget permite mostrar una barra de progreso para la canción y controlar la posición del audio.

```
lib > seek_bar.dart > SeekBar > build
1  import 'package:flutter/material.dart';
2
3  class SeekBar extends StatelessWidget {
4      final Duration duration;
5      final Duration position;
6      final Duration bufferedPosition;
7      final ValueChanged<Duration>? onChangeEnd;
8
9      SeekBar({
10         required this.duration,
11         required this.position,
12         required this.bufferedPosition,
13         this.onChangeEnd
14     });
15
```

Agregando el Slider de progreso

En este paso agregué un Slider dentro del widget SeekBar, el cual muestra el avance de la canción y permite mover la reproducción a otra posición.

```
@override
Widget build(BuildContext context) {
    return Column(
        children: [
            Slider(
                activeColor: Colors.blue,
                inactiveColor: Colors.grey,
                min: 0.0,
                max: duration.inMilliseconds.toDouble(),
                value: position.inMilliseconds.toDouble().clamp(0.0, duration.inMilliseconds.toDouble()),
                onChanged: (value) {
                    onChangeEnd?.call (
                        Duration(milliseconds: value.round())
                    );
                },
            ), // Slider
            Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                    Text(_formatDuration(position), style: const TextStyle(color: Colors.white)),
                    Text(_formatDuration(position), style: const TextStyle(color: Colors.white)),
                ],
            ),
        ],
    );
}
```

Formateando la duración del audio

En este paso agregué una función para convertir la duración del audio a formato de minutos y segundos. Esto permite mostrar el tiempo de reproducción de una forma más clara para el usuario.

```
String _formatDuration(Duration duration) {  
  String twoDigits(int n) =>  
    n.toString().padLeft(2, '0');  
  final minutes =  
    twoDigits(duration.inMinutes.remainder(60));  
  final seconds =  
    twoDigits(duration.inSeconds.remainder(60));  
  return '$minutes:$seconds';  
}
```

Creando la pantalla del reproductor

En este paso creé el archivo `music_player_screen.dart`, donde se importaron las librerías necesarias y se declaró la pantalla principal del reproductor como un `StatefulWidget`, ya que la aplicación necesita actualizarse cuando cambia el estado de reproducción.

```
lib > music_player_screen.dart > MusicPlayerScreen > MusicPlayerScreen
1  import 'package:flutter/material.dart';
2  import 'position_data.dart';
3  import 'seek_bar.dart';
4  import 'package:just_audio/just_audio.dart';
5  import 'package:rxdart/rxdart.dart';
6
7
8  class MusicPlayerScreen extends StatefulWidget {
9    const MusicPlayerScreen({super.key});
10
```

Inicializando el reproductor de audio

En este paso declaré el objeto `AudioPlayer` y lo inicialicé dentro de `initState()`. También se llamó al método encargado de cargar el archivo de audio al iniciar la pantalla.

```
15
16  class _MusicPlayerScreenState extends State<MusicPlayerScreen> {
17    late AudioPlayer _audioPlayer;
18    bool isPlaying = false;
19    String currentTrack = 'Baby';
20    String artist = 'Justin Bieber';
21
22    @override
23    void initState(){
24      super.initState();
25      _audioPlayer = AudioPlayer();
26      _loadAudio();
27    }
28
29    Future<void> _loadAudio() async {
```

Creando el flujo de datos de posición

En este paso creé un flujo llamado `_positionDataStream`, el cual combina la posición actual, la posición en búfer y la duración total de la canción. Esto permite actualizar la barra de progreso mientras el audio se reproduce.

```
Stream<PositionData> get _positionDataStream =>
    Rx.combineLatest3<Duration, Duration, Duration?, PositionData> (
        _audioPlayer.positionStream,
        _audioPlayer.bufferedPositionStream,
        _audioPlayer.durationStream.map((duration) =>
            duration ?? Duration.zero),
        (position, bufferedPosition, duration) =>
            PositionData(position, bufferedPosition, duration!)
    );
```

Agregando la función de reproducir y pausar

En este paso agregué la función `_playPause()`, la cual permite reproducir la canción si está pausada o pausarla si ya se está reproduciendo. También se actualiza la variable `isPlaying` para cambiar el ícono del botón.

```
void _playPause() {
    if(isPlaying) {
        _audioPlayer.pause();
    } else {
        _audioPlayer.play();
    }
    setState(() {
        isPlaying = !isPlaying;
    });
}
```


Diseñando la estructura visual del reproductor

En este paso construí la interfaz principal usando Scaffold, configurando el color de fondo y un AppBar con el título Hear Me!. Esta estructura funciona como base visual para el reproductor.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFF2C2C54),
    appBar: AppBar(
      backgroundColor: const Color(0xFF2C2C54),
      elevation: 0,
      title: const Text(
        'Hear Me!',
        style: TextStyle(color: Colors.white),
      ), // Text
      centerTitle: true,
    ), // AppBar
```

Agregando la imagen de portada

En este paso agregué la imagen del artista dentro de un contenedor circular, utilizando BoxDecoration y AssetImage. Esto permite mostrar una portada visual dentro del reproductor.

```
body: Padding(padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 0),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Container(
        width: 200,
        height: 200,
        decoration: const BoxDecoration(
          shape: BoxShape.circle,
          image: DecorationImage(image: AssetImage('assets/justin.png'),
        ), // DecorationImage
      ), // BoxDecoration
    ], // Container
```

Mostrando el nombre de la canción

En este paso agregué el texto con el nombre de la canción actual, utilizando un widget Text con estilo en color blanco, tamaño grande y letra en negritas.

```
const SizedBox(height: 30,),  
Text(  
  currentTrack,  
  style: const TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.white  
  ) // TextStyle  
) // Text  
const SizedBox(height: 30,),
```

Agregando la barra de reproducción

En este paso agregué un StreamBuilder que recibe los datos del audio y los envía al widget SeekBar. Con esto la barra de progreso puede actualizarse conforme avanza la canción.

```
StreamBuilder<PositionData>(  
  stream: _positionDataStream,  
  builder: (context, snapshot) {  
    final positionData = snapshot.data;  
    return SeekBar(  
      duration: positionData?.duration ?? Duration.zero,  
      position: positionData?.position ?? Duration.zero,  
      bufferedPosition: positionData?.bufferedPosition ?? Duration.zero,  
      onChangeEnd: (newPosition) {  
        _audioPlayer.seek(newPosition);  
      }  
    ); // SeekBar  
  },  
) // StreamBuilder
```

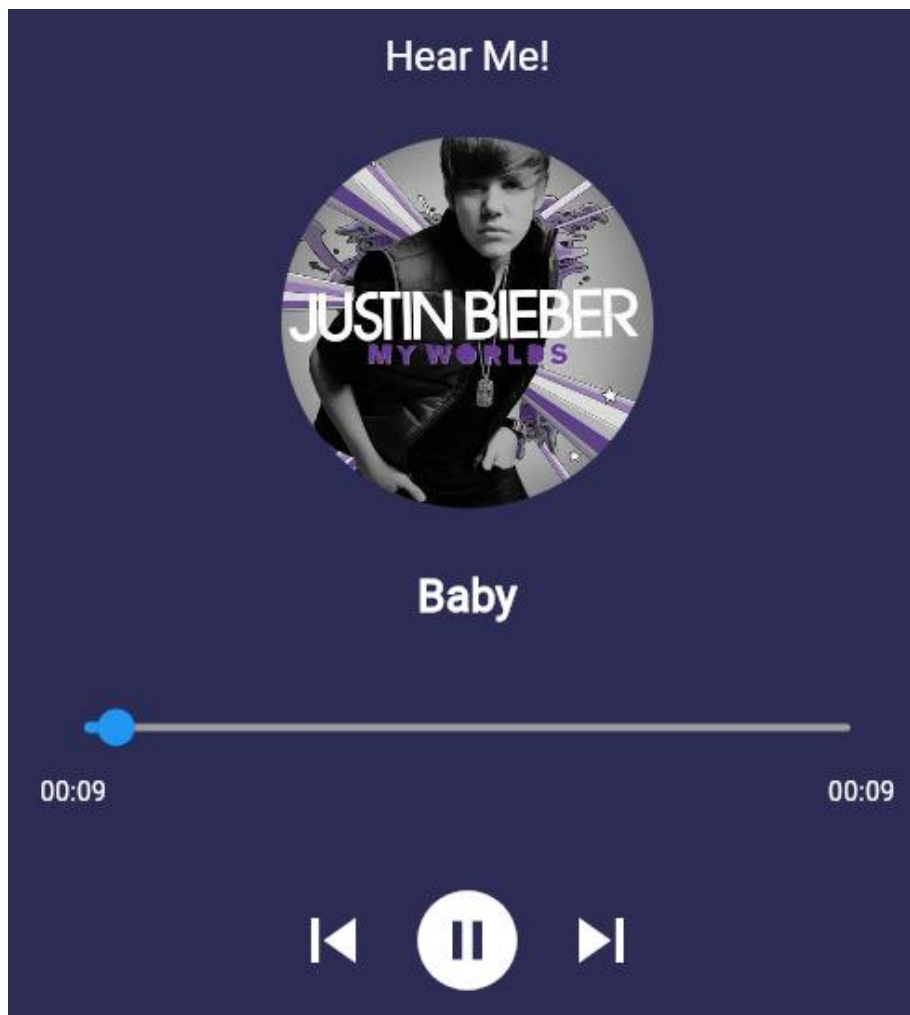
Agregando los botones de control

En este paso agregué los botones de control del reproductor: anterior, reproducir/pausar y siguiente. Estos botones se colocaron dentro de un Row para que aparezcan alineados horizontalmente.

```
const SizedBox(height: 30,),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    IconButton(
      iconSize: 48,
      icon: const Icon(
        Icons.skip_previous,
        color: Colors.white,
      ), // Icon
      onPressed: (){},
    ), // IconButton
    IconButton(
      iconSize: 64,
      icon: Icon(
        isPlaying
          ? Icons.pause_circle_filled
          : Icons.play_circle_fill,
        color: Colors.white,
      ), // Icon
      onPressed: _playPause,
    ), // IconButton
```

```
    IconButton(
      iconSize: 48,
      icon: const Icon(
        Icons.skip_next,
        color: Colors.white,
      ), // Icon
      onPressed: (){},
    ), // IconButton
  ],
) // Row
],
) // Column
), // Padding
); // Scaffold
}
```

Finalmente ejecuté la aplicación para comprobar que el reproductor se mostrara correctamente. La pantalla final muestra el título de la app, la imagen de portada, el nombre de la canción, la barra de progreso y los botones de control.



Preguntas de reflexión

1. ¿Qué retos encontraste al seguir las instrucciones del video?

Uno de los principales retos fue configurar correctamente las dependencias y los archivos dentro del proyecto, ya que era necesario agregar just_audio, rxdart y registrar correctamente los assets en pubspec.yaml. También fue importante revisar errores de sintaxis, como comas, paréntesis e imports, porque un pequeño error impedía que la aplicación pudiera ejecutarse correctamente.

2. ¿Qué diferencia notaste entre una aplicación que simula una interfaz y una que realmente reproduce audio?

La diferencia principal es que una aplicación que solo simula una interfaz únicamente muestra botones, imágenes y texto, pero no realiza una acción real. En cambio, una aplicación que realmente reproduce audio necesita cargar archivos, manejar el estado de reproducción, actualizar la barra de progreso, controlar la pausa y liberar recursos cuando la pantalla deja de utilizarse. Esto hace que la aplicación sea más funcional y no solo visual.

3. ¿Qué widgets consideras más importantes o esenciales para construir una interfaz de un reproductor en Flutter?

Los widgets más importantes para construir un reproductor en Flutter son Scaffold, AppBar, Column, Row, Container, Text, Slider, IconButton y StreamBuilder. Scaffold permite crear la estructura principal, Column y Row organizan los elementos, Container muestra la imagen de portada, Slider representa el avance de la canción, IconButton permite agregar los controles y StreamBuilder ayuda a actualizar la interfaz con los datos reales del audio.